Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2233 Programación Avanzada (2026-1)

Tarea 0

Entrega

- **Fecha y hora oficial:** jueves 19 de marzo de 2026, 20:00.
- **Lugar:** Repositorio personal de GitHub — Carpeta: `Tareas/T0/`
  El código debe estar en la rama (*branch*) por defecto del repositorio: `main`.
- **Pauta de corrección**: en este enlace.
- **Bases generales de tareas (descuentos)** en este enlace.
- **Formulario declaración uso de IA**: en este enlace, disponible hasta 20 de marzo a las 20:00.
- **Ejecución de tarea**: La tarea será ejecutada **únicamente** desde la terminal del computador. Además, durante el proceso de corrección, se cambiará el nombre de la carpeta "`T0/`" por otro nombre y se ubicará la terminal dentro de dicha carpeta antes de ejecutar la tarea. **Los *paths* relativos utilizados en la tarea deben ser coherentes con esta instrucción.**

Objetivos

- Desarrollar algoritmos para la resolución de problemas complejos.
- Aplicar competencias asimiladas en "Introducción a la Programación" para el desarrollo de una solución a un problema.
- Procesar *input* del usuario de forma robusta, manejando potenciales errores de formato.
- Interactuar con diversos archivos de Python y ejecutarlos mediante terminal.
- Trabajar con archivos de texto para leer, escribir y procesar datos.
- Escribir código utilizando paquetes externos (*i.e.* código no escrito por el estudiante), como por ejemplo, módulos que pertenecen a la biblioteca estándar de Python y módulos no estándares.
- Familiarizarse con el proceso de entrega de tareas y uso de buenas prácticas de programación.
- Aplicar conceptos de programación orientada a objetos (POO) para resolver un problema.

Índice

1. *DCColores* **3**
1.1. Reglas . . . 3

2. Flujo del programa **5**
2.1. Parte 1. Funcionalidades . . . 5
2.2. Parte 2. Menú . . . 9
2.2.1. Menú de Juego . . . 9
2.2.2. Menú de Acciones . . . 10
2.3. Formato de entrada y salida . . . 11
2.3.1. Formato de entrada de tablero . . . 11
2.3.2. Formato de guardado de listado de cambios . . . 11

3. Archivos **12**
3.1. Código inicial . . . 12
3.2. `visualizador.pyc` . . . 13

4. `.gitignore` **13**

5. Importante: Corrección de la tarea **14**
5.1. Ejecución de *tests* . . . 14

6. Restricciones y alcances **15**

1. *DCColores*

La NASA llegó a marte y quiere terraformar la superficie del planeta modificando la altitud del terreno estratégicamente. Para esto, desarrollaron un laser terraformador que eleva o hunde el terreno en el área afectada.

Desde la NASA contrataron al DCC para desarrollar un programa que determine las posiciones a las que hay que apuntar el laser para lograr el resultado deseado.

1.1. Reglas

DCColores es un juego individual que consiste en un conjunto de tableros que el jugador debe resolver.

Cada tablero es una cuadrícula rectangular de $N \times M$ celdas, donde algunas de estas celdas son capaces de cambiar de color y otras no.

Para representar el estado de cada unas de estas celdas se utilizarán números. Las celdas que cambian de color varían entre dos estados, por lo que se representará su color con los números `1` y `-1`, mientras que las que no cambian de color serán representadas por un `0`.

-1	1	-1	0
1	-1	1	-1
0	1	-1	1

Figura 1: Ejemplo de un tablero de 3×4.

El tablero utiliza un sistema coordenado para identificar las celdas, donde el primer valor representa la fila y el segundo valor la columna correspondiente a cada celda. El origen de este sistema de coordenadas (la coordenada $(0, 0)$) corresponde a la celda ubicada en la esquina superior izquierda de cada tablero.

Para modificar las celdas del tablero se deberá aplicar un **cambio** en una de esta. Cada cambio modifica el color de la celda a la que se encuentra asociada y las celdas colindantes a esta, tanto de forma vertical, como horizontal y diagonal.

1	-1	1	0
-1	1	-1	-1
0	-1	1	1

(a) Cambio solo en la celda $(1, 1)$

-1	1	-1	0
1	-1	-1	1
0	1	1	-1

(b) Cambio solo en la celda $(2, 3)$

Figura 2: Ejemplo de distintos cambios aplicados sobre el tablero de la Figura 1.

Un cambio puede estar asociado a la una celda ubicada tanto en el borde del tablero (Figura 2a) como en el interior de este (Figura 2b), la única condición es que la celda asociada debe ser una celda capaz de cambiar de color. Por lo mismo, en el tablero de la Figura 1 no sería posible que un cambio esté asociado a las celdas de las coordenadas $(2, 0)$ y $(0, 3)$.

Para resolver un tablero se te entregará un estado inicial y una meta a la que se debe llegar. A partir de estos, deberás obtener un listado de cambios los cuales consisten en un conjunto de coordenadas que se deben ejecutar.

0	0	1	0	0
0	1	0	1	0
1	1	1	1	1
0	1	0	1	0
0	0	1	0	0

(a) Estado inicial.

0	0	-1	0	0
0	1	0	-1	0
-1	1	-1	1	-1
0	-1	0	1	0
0	0	-1	0	0

(b) Meta esperada.

Figura 3: Ejemplo de un tablero de 5×5 y su respectiva meta.

Posibles soluciones al caso presentado en la Figura 3 serían:

- $(1, 1), (2, 2), (3, 3)$
- $(0, 2), (0, 2), (2, 2), (3, 3), (1, 1)$
- $(4, 2), (3, 3), (4, 2), (2, 2), (4, 2), (1, 1), (4, 2)$
- $(3, 3), (1, 1), (3, 3), (2, 2), (3, 3)$
- Infinitas otras soluciones

2. Flujo del programa

Para esta tarea se deben completar dos partes:

Parte 1 Completar el modelamiento de las clases entregadas, llamadas `DCColores` y `Tablero`. Estas clases modelarán respectivamente el juego completo, y un tablero particular. La clase `DCColores` debe mantener una lista de `Tableros`, los cuales deben poder ser leídos y escritos desde/hacia archivos, además de mantener el estado del usuario actual. Por otro lado, la clase `Tablero` debe permitir manipular los estados de las casillas del tablero, determinar si el estado del tablero es una solución, determinar si una solución existe y resolver el tablero automáticamente.
Esta parte será corregida automáticamente mediante el uso de *tests*.

Parte 2 Construir un menú por consola, que permita que el jugador interactúe con *DCColores*.
Esta parte será corregida manualmente por el cuerpo docente.

Sobre los *tests*

Junto con el código base, se entregará un conjunto de *tests* que les permitirán verificar distintos casos en base a su respuesta esperada. Este conjunto de *tests* serán los "***tests* públicos**". Los *tests* públicos evalúan distintos casos de uso del programa, pero no necesariamente cubren todos los casos posibles. En la etapa de corrección, el equipo docente usará un conjunto de *tests* distintos que serán los ***tests* privados** y contienen *tests* que pueden evaluar *inputs* distintos y casos que pueden no haber sido incluidos en los *públicos*. Posteriormente a la entrega, se publicarán también los *tests* privados de manera que puedan corroborar su puntaje.

Por lo tanto, **es responsabilidad del estudiantado confeccionar una solución que cumplan con lo expuesto en el enunciado y que no esté creada solamente a partir de los *tests* públicos**. De ser necesario, el estudiantado deberá pensar en nuevos casos que sean distintos a los *tests* públicos. **No se aceptarán supuestos que funcionaron en los *tests* públicos, pero que van en contra de lo expuesto en el enunciado.**

Para cada clase y método indicado en el enunciado, **no puedes modificar su nombre, agregar nuevos argumentos o argumentos por defecto**. Tampoco está permitido cambiar el nombre y ubicación de los archivos entregados. En caso de no respetar esto, se aplicará un descuento importante. Puedes crear nuevos atributos, nuevos métodos, archivos y/o funciones si estimas conveniente. También se permite crear funciones en otro módulo y que el método que pedimos completar únicamente llame a esa función externa. Sin embargo, para que los *tests* funcionen correctamente, los métodos y atributos explícitamente indicados aquí deben respetarse.

2.1. Parte 1. Funcionalidades

En esta parte se debe completar el modelamiento de las clases `Tablero` y `DCCasillas`, para las cuales se entrega un código base inicial.

Al completar las clases solicitadas, ten en consideración los siguiente:

- Al completar los atributos como mínimo debes considerar los que se indica a continuación. Si lo consideras necesario, puedes puedes agregar los atributos adicionales que estimes conveniente.
- Al completar cada métodos no debes modificar el nombre ni la cantidad de parámetros que recibe.

Modificar `class Tablero:`

Clase que representa un tablero del juego *DCColores*.

- Modificar `def __init__(self, inicio: list[list[int]], meta: list[list[int]]) -> None:`

  Inicializa una instancia de tablero. Inicializa al menos los siguientes atributos:

  - `self.inicio` Estado inicial del tablero. Consiste en una lista de lista de números.
  - `self.meta` Estado final deseado. Consiste en una lista de lista de números.

- Modificar `def cambiar_color(self, tablero_actual: list[list[int]], x: int, y: int) -> list[list[int]]:`

  Recibe el estado actual del tablero y las coordenadas `x` e `y` de una celda. Según lo indicado en la Subsección 1.1: Reglas, cambia el color de la celda recibida y sus celdas colindantes. Retorna el tablero actualizado.

- Modificar `def aplicar_cambios(self, cambios: list[tuple[int]]) -> list[list[int]]:`

  Recibe una lista de cambios a aplicar sobre el tablero inicial. Cada cambios corresponde a un par de números que representa una coordenada del tablero. Retorna el estado final del tablero inicial tras haber aplicado todos los cambios sobre el mismo.

  Debes asegurar que este método no modifique el contenido del atributo `self.inicial`. Para asegurar lo anterior, recomendamos investigar sobre la librería `deepcopy`.

- Modificar `def validar_solucion(self, cambios: list[tuple[int]]) -> bool:`

  Recibe un listado de cambios y comprueba si estos cambios permiten llegar hasta la meta deseada o no. Retorna un `bool` que indica lo anterior.

- Modificar `def encontrar_solucion(self, cambios: list[tuple[int]], cambios_adicionales: int) -> list[tuple[int]]:`

  Recibe un listado de cambios y una cantidad máxima de cambios adicionales que se pueden agregar. A partir del listado de cambios y el estado inicial del tablero, calcula el estado actual del tablero y, considerando el máximo de cambios a agregar, encuentra y retorna un listado de cambios que permite llegar al tablero final deseado. El listado a retornar debe considerar los cambios recibidos como *input*.

  Por ejemplo, a continuación se presenta el estado inicial y meta de un tablero (Figura 4a y 4b resp.). Considerando que se llama el método `encontrar_solucion` y se le entrega el listado de cambios $[(1,0),(2,1)]$, entonces se quiere encontrar un listado de cambios que permita llegar hasta la meta desde el tablero Figura 4d.

1	1	1
1	1	1
1	1	1

(a) Estado inicial

1	-1	1
-1	1	-1
1	-1	1

(b) Meta esperada

-1	-1	1
-1	-1	1
-1	-1	1

(c) Estado después de aplicar cambio en celda $(1, 0)$

-1	-1	1
1	1	-1
1	1	-1

(d) Estado después de aplicar los cambios $(1, 0)$ y $(2, 1)$

Figura 4: Ejemplo de un tablero de 3×3 con solución.

Si tienes 3 cambios adicionales para llegar hasta la meta, los posibles listados de cambios que puede retornar este método son:

- $[(1, 0), (2, 1), (0, 1), (1, 2)]$
- $[(1, 0), (2, 1), (1, 2), (0, 1)]$

En caso de que no encontrar un conjunto de cambios que permita llegar a la meta en el límite de cambios indicado, entonces retorna una lista vacía.

Por ejemplo, un tablero que nunca podrá encontrar una solución, independientemente de la límite de cambios, sería el siguiente:

1	1
1	1

(a) Estado inicial

-1	1
1	-1

(b) Meta esperada

Figura 5: Ejemplo de un tablero de 2×2 sin solución.

Hint: Si bien existen diversas maneras de encontrar de solucionar un tablero, o bien de determinar que no es posible encontrar una solución, la técnica de *backtracking* puede ser particularmente útil en este caso.

- Modificar `def optimizar_solucion(self, cambios: list[tuple[int]]) -> list[tuple[int]]:`

Muchas veces las soluciones obtenidas por `encontrar_solucion` pueden ser redundantes y contener cambios que son anulados por los cambios que ocurren después.

Por ejemplo, revisando las soluciones que fueron planteadas anteriormente para la Figura 3:

Solución original	Solución optimizada
$(1,1),(2,2),(3,3)$	$(1,1),(2,2),(3,3)$
$\mathbf{(0,2)},\mathbf{(0,2)},(2,2),(3,3),(1,1)$	$(2,2),(3,3),(1,1)$
$\mathbf{(4,2)},(3,3),\mathbf{(4,2)},(2,2),\mathbf{(4,2)},(1,1),\mathbf{(4,2)}$	$(3,3),(2,2),(1,1)$
$\mathbf{(3,3)},(2,2),\mathbf{(3,3)},(1,1),\mathbf{(3,3)}$	$(2,2),(1,1),\mathbf{(3,3)}$

Por lo tanto, este método recibe un listado de cambios y retorna un nuevo listado de cambios en el que se han eliminado los cambios redundantes.

Modificar `class DCColores:`

Clase que representa el juego. Al crear un juego, se carga una serie de tableros. Los tableros son independientes unos de otros, por lo que el jugador puede jugar los tableros en cualquier orden.

- Modificar `def __init__(self) -> None:`

  Inicializa una instancia del juego. El juego debe inicializar al menos los siguientes atributos:

  - `self.tableros` Una `list` de instancias de `Tablero` que contiene todos los tableros disponibles en el juego. Se inicializa a partir de los archivos contenidos en el directorio `data/`, cuyo formato se describe en Subsección 2.3: Formato de entrada y salida.
  - `self.tablero_actual` Un `int` correspondiente al índice del `Tablero` de la lista `self.tableros` que actualmente se está resolviendo. En caso de que no haya un tablero seleccionado, su valor debe ser `-1`.
  - `self.cambios_actuales` Lista de cambios que actualmente se están aplicando sobre el tablero actual. En caso de que no haya un tablero seleccionado o que no se hayan realizado cambios en el tablero, su valor debe ser una lista vacía.

- Modificar `def cargar_tablero(self, archivo_tablero: str) -> None:`

  Recibe un texto con el nombre del archivo que contiene el tablero. Crea una instancia de `Tablero` con la información correspondiente y la agrega al final de la lista `self.tablero`.

  Puedes asumir que el archivo recibido siempre existirá y que su formato cumple con lo descrito en la Subsección 2.3.

- Modificar `def guardar_cambios(self, nombre_archivo: str) -> None:`

  A partir del nombre de archivo recibido y el listado de cambios actuales, escribe dicha solución en el directorio `data/` siguiendo el formato descrito en Subsección 2.3. Si ya existe una archivo con ese nombre, se sobrescribe su contenido.

2.2. Parte 2. Menú

En esta parte deberás implementar un menú que permita interactuar con el juego *DCColores*. Para esto, se ejecutará tu programa mediante un archivo `main.py`. Las acciones a ejecutar en los menús deberán aceptar entradas de usuario mediante terminal.

Se deben crear al menos dos menúes: Menú de Juego y Menú de Acciones. En esta parte de la tarea, se evaluará principalmente que (1) ambos menús sean a prueba de errores, (2) ambos incluyan al menos todas las opciones solicitadas, (3) que los parámetros recibidos por el usuario se manejen de forma correcta, y (4) que cada opción del menú llame, mediante código, a la función o método correspondiente.

En esta parte únicamente se evaluará el correcto funcionamiento de los menú. Se espera que los un correcto manejo de *inputs*, asegurando que los menús sean a prueba de errores.

2.2.1. Menú de Juego

Este es el menú encargado de iniciar la interacción con el usuario y realizar la carga de *DCColores*. A continuación se entrega un ejemplo de cómo se podría ver el menú de inicio tras haber ejecutado por primera vez `python3 main.py` :

```
¡Bienvenido a DCColores!

*** Selecciona un tablero ***

[1] Tablero 4x6
[2] Tablero 5x5
[3] Tablero 3x3
[0] Salir del programa

Indique su opción (1, 2, 3, 0):
```

Figura 6: Ejemplo de Menú de Inicio

El menú inicio debe listar todos los tableros disponibles en el directorio `data/`, queda a tu criterio como mostrarlo en el listado y permitir que el usuario los distinga. Además, debe permitir que el usuario seleccione uno de los tableros disponibles. Si el usuario elige un tablero válido, debe pasar al Menú de Acciones. En caso contrario, se debe comunicar el error y mantener el menú actual.

2.2.2. Menú de Acciones

Luego de haber seleccionado exitosamente un tablero desde el Menú de Juego, se abrirá un segundo menú encargado de realizar las acciones sobre los tableros. Se debe indicar cuál es el tablero actual, y ofrecer las siguientes acciones:

```
Tablero 3x5

Meta:
##O##
_OOO_
##O##

Estado actual:
OO###
_#O#_
OOO##

Cambios: (0, 0), (2,1)

Solución aún no encontrada

*** Menú de Acciones ***

[1] Agregar cambio
[2] Deshacer cambio
[3] Encontrar solución
[4] Guardar cambios
[0] Volver al menú de Juego

Indique su opción (1, 2, 3, 4, 0):
```

Figura 7: Ejemplo de Menú de Acciones

El menú debe mostrar: la meta a la que se debe llega, el estado actual del tablero, el listado de cambios actuales, e indicar si el estado actual corresponde a la solución o no. La forma de representar el tablero se explicará con detalle en la Subsección 3.2: `visualizador.pyc`.

Las acciones del menú de acciones deben ser las siguientes:

1. **Agregar cambio**: Pide al usuario las coordenadas de una celda y la agrega al listado de cambios. Si se intenta agregar al listado una celda no permitida, debe indicar el error y continuar.

   Luego de ejecutar la acción, vuelve a cargar el Menú de Acciones con la información actualizada.

2. **Deshacer cambio**: Elimina el último cambio agregado al listado de cambios actuales.

   Luego de ejecutar la acción, vuelve a cargar el Menú de Acciones con la información actualizada.

3. **Encontrar solución**: Pide al usuario una límite de cambios. A partir del tablero actual, el listado de cambios actuales y el límite de cambios indicado por el usuario, busca una solución optimizada y le avisa al usuario si fue posible encontrar una solución o no. En caso de haber encontrado una solución, se actualiza el listado de cambios actuales.

   Luego de ejecutar la acción, vuelve a cargar el Menú de Acciones con la información actualizada.

4. **Guardar cambios**: Solicita al usuario un nombre de archivo y guarda la solución en el archivo correspondiente.

5. **Volver al menú de juego**: Regresa al menú de juego.

2.3. Formato de entrada y salida

Aquí se describen los formatos de los archivos de entrada y salida del programa. Todos los archivos deben considerarse como archivos de texto en codificación `UTF-8`.

2.3.1. Formato de entrada de tablero

Para describir un tablero se utiliza un archivo de extensión `.brd`, donde la primera línea contiene dos enteros N y M separados por un espacio, que indican respectivamente la cantidad de filas y cantidad de columnas de celdas.

A continuación se encontrarán $2 \times N$ líneas. Las primeras N líneas corresponden a las filas iniciales del tablero, mientras que las siguientes N líneas corresponden a las filas de la meta. Dentro de cada fila hay M caracteres que representa el valor de cada celda:

- `_`: Celda que no cambia de color.
- `#`: Celda que sí cambia de color en estado `1`.
- `O`: Celda que sí cambia de color en estado `-1`.

Por ejemplo, para el tablero de ejemplo de la izquierda, el archivo de entrada correspondiente se muestra a la derecha:

```
3 5
#####
_###_
#####
OO#OO
_###_
OO#OO
```

(a) Archivo `.brd`

1	1	1	1	1
0	1	1	1	0
1	1	1	1	1

(b) Estado inicial

-1	-1	1	-1	-1
0	1	1	1	0
-1	-1	1	-1	-1

(c) Meta esperada

Figura 8: Ejemplo archivo de carga, junto a su respectivo estado inicial y meta.

2.3.2. Formato de guardado de listado de cambios

Para guardar el estado de un juego, se utiliza un archivo de extensión `.chg`. En la primera línea contiene tres enteros N, M y C separados por un espacio, que indican respectivamente la cantidad de filas, columnas de celdas del tablero y la cantidad de cambios que guarda el archivo. A continuación, se presenta N filas con el estado inicial del tablero utilizando la modo texto del la función `tablero_a_texto`.

Después, por cada cambios que contiene el archivo se presenta: primero una línea con dos enteros X e Y que corresponden a las coordenadas de la celda donde se aplica el cambio; a continuación N filas con el estado del tablero tras aplicar el cambio, debes utilizar la función `tablero_a_texto` en modo texto. Finalmente, al final del archivo se debe dejar una línea en blanco.

Por ejemplo, en la Figura 9 se muestra el estado inicial de un tablero y la meta esperada, donde se necesita de 2 cambios para resolver el tablero: $(0, 4), (2, 0)$. A partir de lo anterior se genera el archivo `.chg`.

-1	-1	1	1	1
0	-1	1	-1	0
1	1	1	-1	-1

(a) Estado inicial

-1	-1	1	-1	-1
0	1	1	1	0
-1	-1	1	-1	-1

(b) Meta esperada

```
3 5 2
OO###
_O#O_
###OO
0 4
OO#OO
_O##_
###OO
2 0
OO#OO
_###_
OO#OO
```

(c) Archivo `.chg`

Figura 9: Archivo de guardado para un tablero con 2 cambios.

3. Archivos

Aquí describe la estructura base del código de la tarea, y los archivos relevantes que deben estar en su repositorio.

3.1. Código inicial

Se te entregarán algunos archivos que deberás completar. Puedes crear más archivos si lo estimas conveniente.

- **Crear** `main.py`: Archivo principal que ejecuta el menú por la consola.
- **Modificar** `dccolores.py`: Contiene la definición de la clase `DCColores`.
- **Modificar** `tablero.py`: Contiene la definición de la clase `Tablero`.
- **No modificar** `visualizador.pyc`: Este archivo contiene la función `tablero_a_texto` que deberás utilizar para mostrar un tablero en la consola. Su uso se describe en la sección `visualizador.pyc`.
- **No modificar** `data/`: Esta carpeta contendrá archivos de extensión `.brd` y `.chg` que almacenan estados del juego. Estos archivos usan el formato indicado en Subsección 2.3: Formato de entrada y salida.
- **No modificar** `tests_publicos/`: Esta carpeta contendrá una serie de archivos `.py` que corresponden a diferentes *tests* para apoyar el desarrollo de la Parte 1. Puedes utilizar los archivos entregados para ir viendo si lo desarrollado hasta el momento cumple con lo esperado en esta evaluación. **Importante**: los *tests* entregados en esta carpeta no serán los mismos que se utilizarán para la corrección de la evaluación.
- **Modificar** `README.md`: Contendrá inicialmente las actualizaciones a la tarea aplicadas hasta el momento. Debe modificarse para personalizar tu propio `README`.
- **No modificar** `README_inicial.md`: Es la base desde la cual deberás construir tu propio `README`.

3.2. `visualizador.pyc`

Para facilitar tu trabajo y evaluar el uso de módulos, se te hará entrega del módulo `visualizador.pyc` que contiene la función `tablero_a_texto` que deberás utilizar en la tarea. Este archivo se encuentra compilado, por lo que no se puede visualizar su contenido, y funciona únicamente con la versión de Python del curso: 3.12.X. Utilizar una versión distinta a 3.12 provocará un error de **magic number**. Este archivo se importa como cualquier módulo, por ejemplo con `import visualizador`.

La función que contiene es: `tablero_a_texto(tablero: list[list[int]], modo: str) -> str`. Recibe una lista de listas de números con el estado de un tablero y un *string* que indica el modo de representación del tablero. Entrega la representación como texto de un tablero recibido.

Los modos de representación son:

1	-1	0
0	1	-1
-1	0	1

Tablero a representar

(a) Modo `color`

(b) Modo `emoji`

```
#O_
_#O
O_#
```

(c) Modo `texto`

Dado que estos modos de presentación pueden variar su comportamiento en cada computador, eres libre de utilizar el modo de representación que más te acomode.

Debes importar correctamente este archivo y hacer uso de su función para el desarrollo de la tarea.

4. `.gitignore`

Para esta tarea **deberás utilizar un** `.gitignore` para ignorar los archivos indicados, este deberá estar dentro de tu carpeta `Tareas/T0/`.

Los elementos que no debes subir y **debes ignorar mediante** `.gitignore` para esta tarea son:

- El enunciado.
- El archivo `visualizador.pyc`
- La carpeta `data/` y todo lo contenido en ella.
- La carpeta `tests_publicos/` y todo lo contenido en ella.
- El archivo `README_inicial.md`.

Recuerda **no ignorar archivos vitales de tu tarea como los que tú debes modificar, o tu tarea no podrá ser revisada**. Es importante que hagan un correcto uso del archivo `.gitignore`, es decir, los archivos **deben** no subirse al repositorio debido al uso correcto del archivo `.gitignore` y no debido a otros medios.

5. Importante: Corrección de la tarea

Para esta tarea, el carácter funcional del programa será el pilar de la corrección, es decir, **sólo se corrigen tareas que se puedan ejecutar**. Por lo tanto, se recomienda hacer periódicamente pruebas de ejecución de su tarea y *push* en sus repositorios.

En el **siguiente enlace** se encuentra la distribución de puntajes. En color **amarillo** se encuentra cada ítem que será evaluado a nivel funcional y de código, es decir, aparte de que funcione, se revisará que el código esté bien confeccionado y que la funcionalidad esté correctamente integrada en el programa. Todo aquel que no esté pintado de amarillo significa que será evaluado si y sólo si se puede probar con la ejecución de su tarea o con los *tests*.

Importante: Todo ítem corregido por el cuerpo docente será evaluado únicamente de forma ternaria: cumple totalmente el ítem, cumple parcialmente o no cumple con lo mínimo esperado. Del mismo modo, todo ítem corregido automáticamente será evaluado de forma ternaria: puntaje completo si pasa todos los *tests* de dicho ítem, medio punto para quienes pasan más del 70 % de los *test* de dicho ítem, y 0 puntos para quienes no superan el 70 % de los *tests* en dicho ítem. Finalmente, todos los descuentos serán asignados manualmente por el cuerpo docente respetando lo expuesto en **el documento de bases generales**.

La corrección se realizará en función del último *commit* realizado antes de la fecha oficial de entrega. Si durante la realización de tu tarea se te presenta algún problema o situación que pueda afectar tu rendimiento, no dudes en contactar al ayudante de Bienestar de tu sección. El correo está en el **siguiente enlace**.

5.1. Ejecución de *tests*

Para la corrección automática se entregarán varios archivos `.py` los cuales contienen diferentes *tests* que ayudan a validar el desarrollo de la Parte 1. Funcionalidades. Para ejecutar estos *tests*, primero debes posicionar tu terminal/consola en la carpeta de la tarea `Tareas/T0/`. Luego, desde esta misma, debes escribir el siguiente comando para ejecutar todos los *tests*:

- `python3 -m unittest discover tests_publicos -b`

En cambio, si deseas ejecutar un subconjunto de *tests*, puedes hacerlo escribiendo lo siguiente:

- `python3 -m unittest tests_publicos.<test_N> -b`
  Reemplazando `<test_N>` por el test que desees probar.

Por ejemplo, si quisieras probar si realizaste correctamente el método `modificar_casilla` de `Tablero`, deberás escribir lo siguiente:

- `python3 -m unittest tests_publicos.test_00_iniciar_tablero -b`

Si lo consideras necesario, puedes agregar al final de los comandos anteriores la *flag* `-v` para ejecuta los *tests* con un mayor nivel de verbosidad. Un mayor nivel de verbosidad permite que se muestre el detalle (nombre e identificador) de todos test ejecutados. Si no se entrega esta *flag*, solo se muestra el detalle de los *tests* que fallan su ejecución.

Haciendo que los comandos anteriores se vean:

- `python3 -m unittest discover tests_publicos -b -v`
- `python3 -m unittest tests_publicos.test_00_iniciar_tablero -b -v`

Importante: recuerda que si `python3` no funciona, probar con el comando específico de tu computador. Este puede ser `py`, `python`, `py3` o `python3.12`.

6. Restricciones y alcances

- Esta tarea es **estrictamente individual**, y está regida por el **Código de honor de Ingeniería**.
- Tu programa debe ser desarrollado en Python 3.12.X con X mayor o igual a 0.
- Tu programa debe estar compuesto por uno o más archivos de extensión `.py` que estén correctamente ordenados por carpeta. **No se revisará archivos en otra extensión como `.ipynb`**.
- Si no se encuentra especificado en el enunciado, supón que el uso de cualquier librería Python **está prohibido**. Pregunta en la *discussion* especial del **foro** si es que es posible utilizar alguna librería en particular.
- Debes adjuntar un archivo `README.md` **conciso y claro**, donde describas los alcances de tu programa, cómo correrlo, las librerías usadas, los supuestos hechos, y las referencias a código externo. El no incluir este archivo o bien incluirlo pero que se encuentre vacío conllevará un **descuento** en tu nota.
- Esta tarea se debe desarrollar **exclusivamente** con los contenidos liberados al momento de publicar el enunciado. No se permitirá utilizar contenidos que se vean posterior a la publicación de esta evaluación.
- Se encuentra estrictamente prohibido citar código que haya sido publicado **después de la liberación del enunciado**. En otras palabras, solo se permite citar contenido que ya exista previo a la publicación del enunciado.
- Cualquier aspecto no especificado queda a tu criterio, siempre que no pase por sobre otro que sí sea especificado por enunciado o por lo evaluado en los *tests* públicos.

Las tareas que no cumplan con las restricciones del enunciado obtendrán la calificación mínima (1,0)**.**